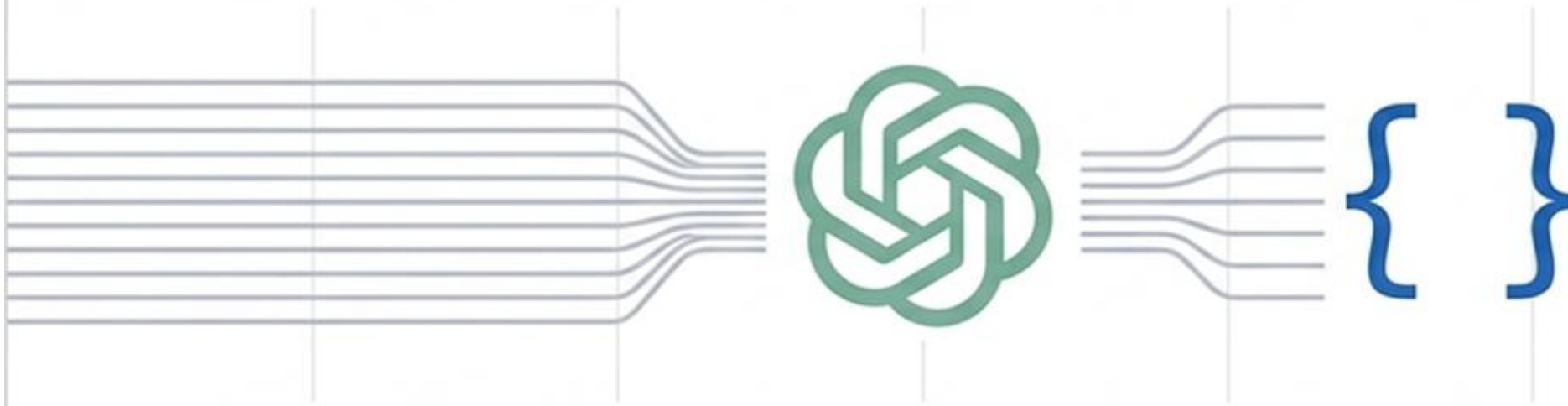
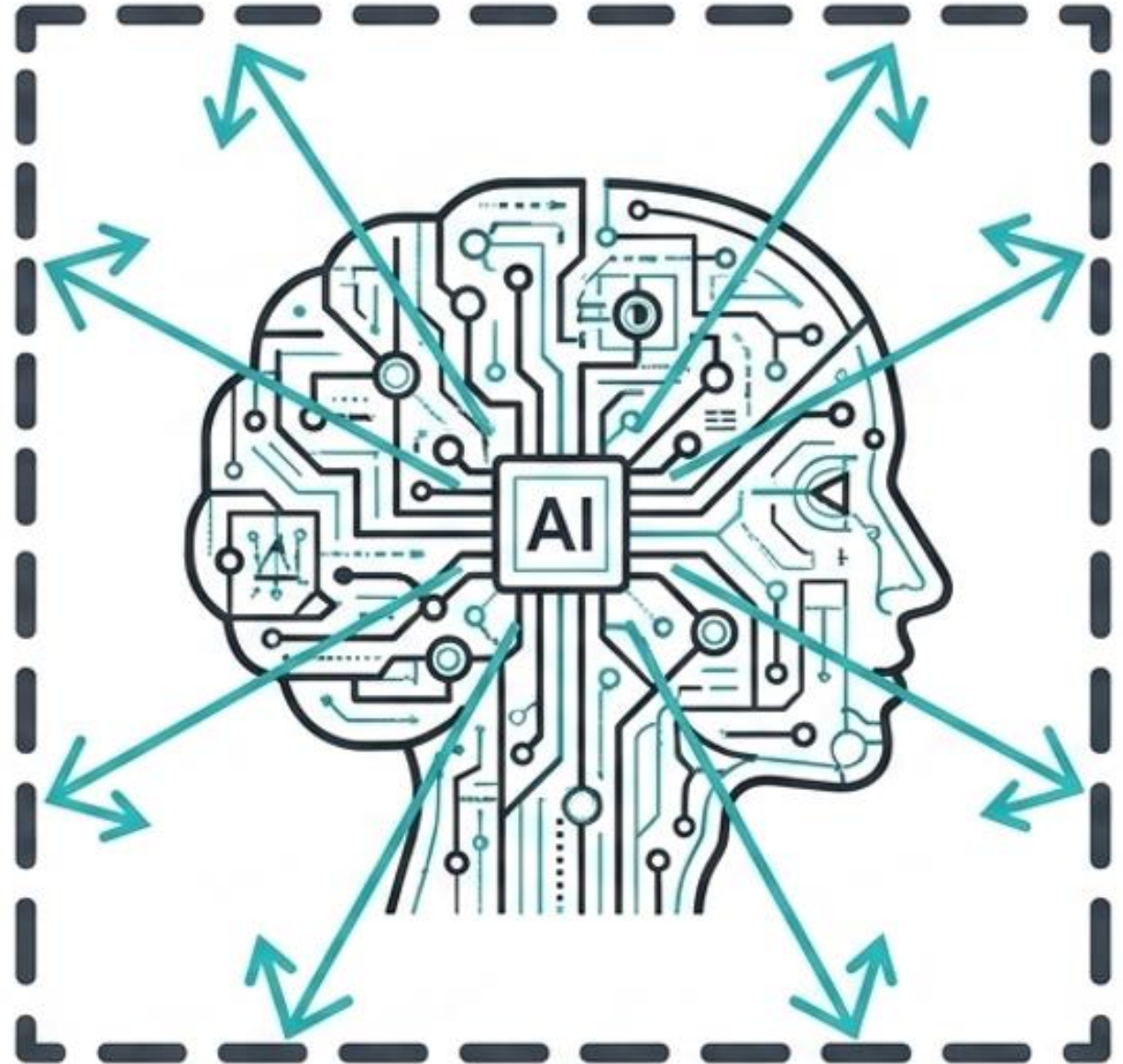


OpenAI APIs 101: A Developer's Quick Start



Agenda

- OpenAI API and Its Ecosystem
- Responses API
 - Quick Start
 - Directing Model Behavior
 - Conversation State
 - Function Calling
- Hand On Demo



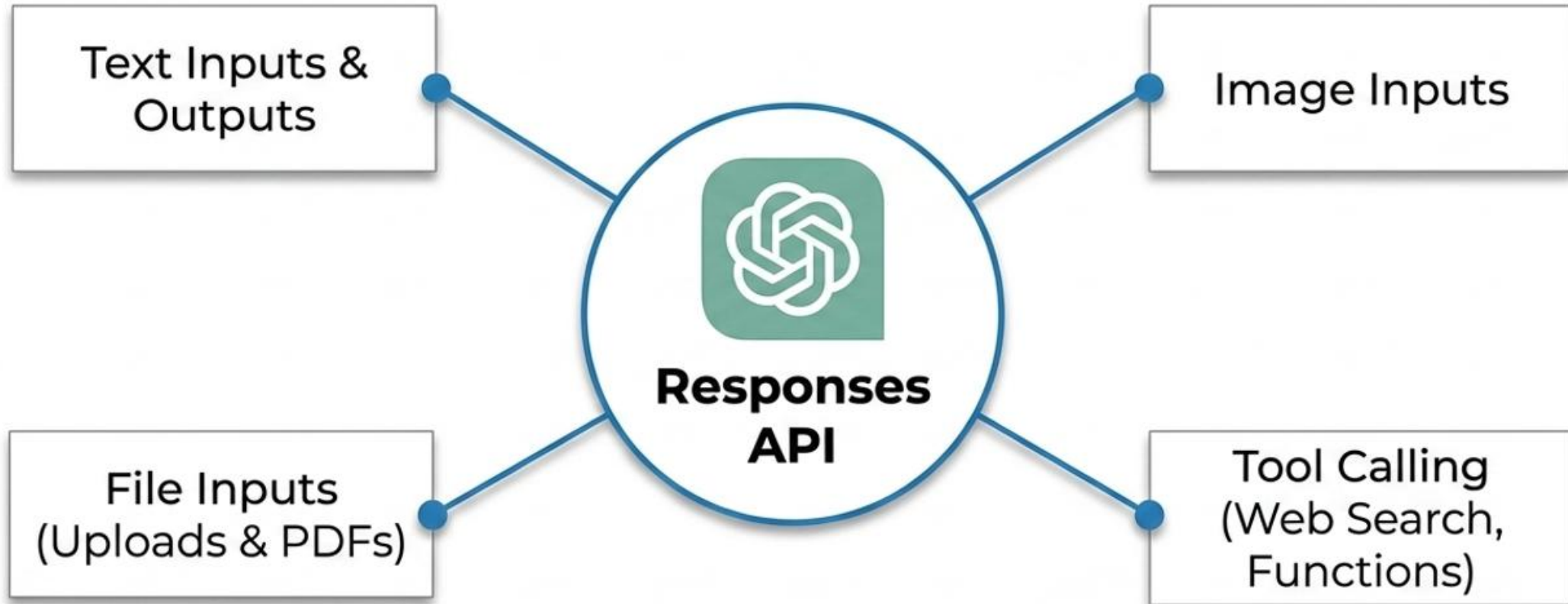
OpenAI API And Its Ecosystem

- **OpenAI API** is an API that provide multiple capabilities to interact with advanced **AI models** from OpenAI.
 - **AI Model:** A computational system designed to perform tasks that typically require human intelligence, such as:
 - **Language understanding**
 - **Image recognition**
 - **Speech to text**
 - **Code generation**
 - **Reasoning and decision making**

OpenAI API And Its Ecosystem

- **OpenAI APIs** provide access to **Large Language Models (LLMs)**:
 - **LLM**: A model trained to understand, generate, and reason with human language.
- Examples of **OpenAI LLMs**:
 - GPT-5.4
 - GPT-5.2
 - GPT-4o-mini
 - GPT-4.1-nano
- **ChatGPT** is a web application powered by OpenAI's modern AI infrastructure:
 - Built on the **Responses API** and Assistants architecture
 - Supports **text, images, files, tools, and voice**
 - **Free and paid tiers** dynamically use **GPT-5.*** class models

OpenAI Responses API



STATEFUL INTERACTIONS: The API can use the output of previous responses as new inputs, maintaining conversational state and memory.

Getting Started With OpenAI API &



- **Create** and **export** an API key
- **Install** the OpenAI SDK
- **Run** a basic API request
- Analyze **images** and **files**
- Extend the model with **tools**

Create And Export An API Key

- Create an [API key](#) in the [OpenAI developer platform](#)

Create new secret key

Owned by

You Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

My Test Key

Project

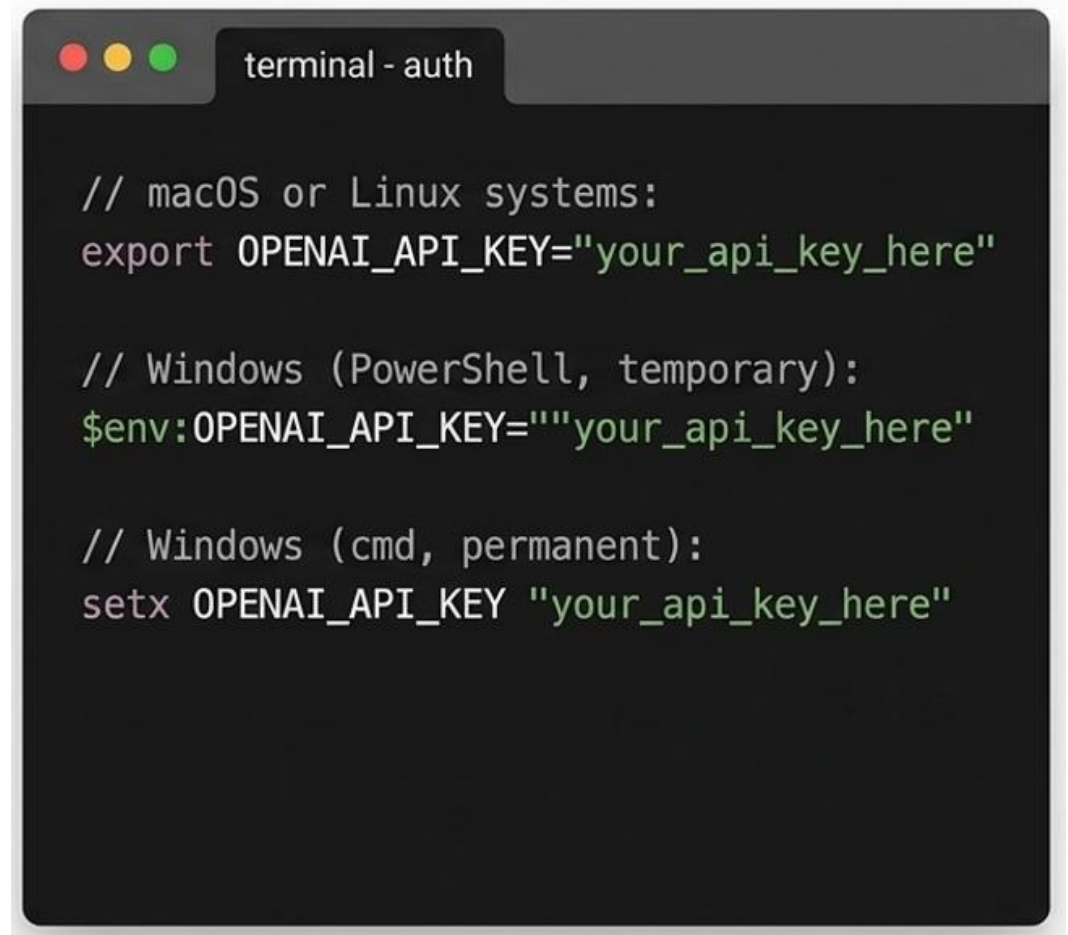
Permissions

All Restricted Read only

Cancel

Create secret key

- Export the key to the **OPENAI_API_KEY** environment variable



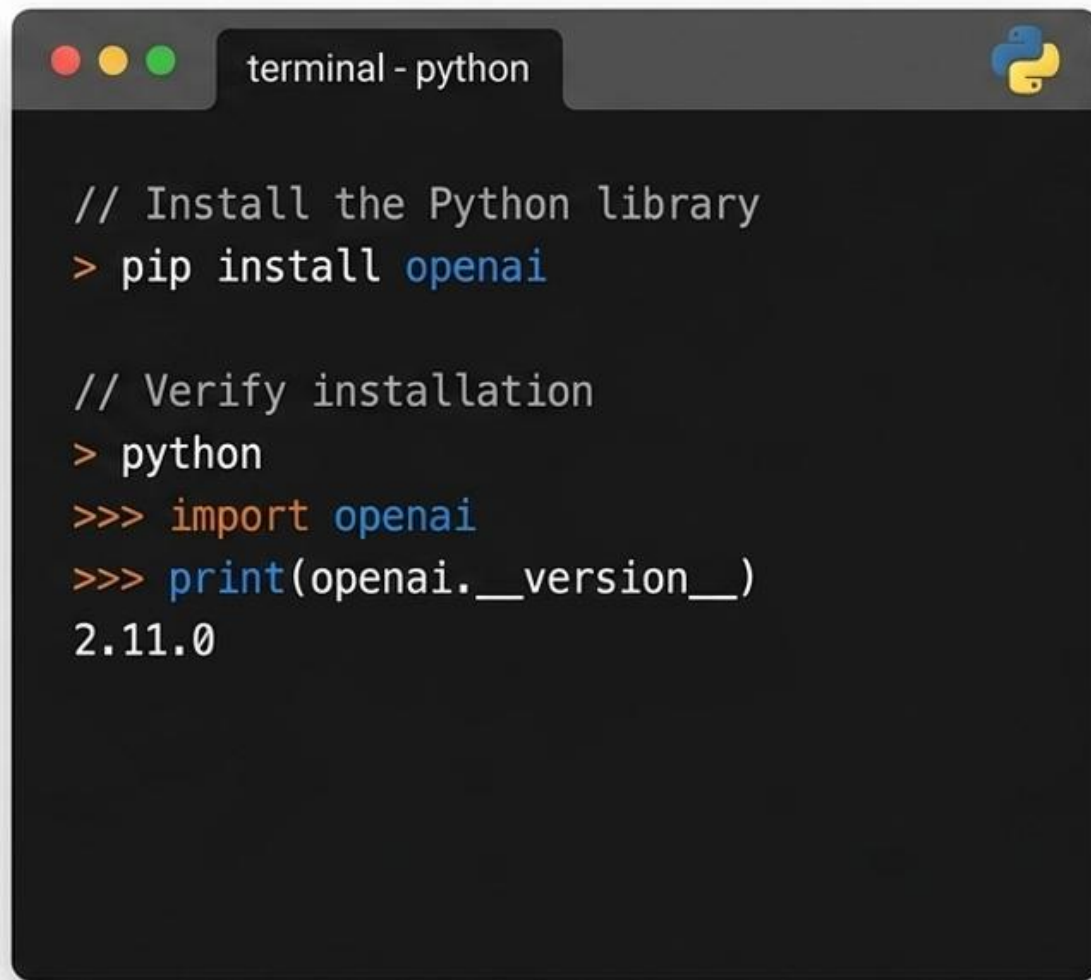
```
terminal - auth

// macOS or Linux systems:
export OPENAI_API_KEY="your_api_key_here"

// Windows (PowerShell, temporary):
$env:OPENAI_API_KEY="your_api_key_here"

// Windows (cmd, permanent):
setx OPENAI_API_KEY "your_api_key_here"
```

Install & Verify The OpenAI SDK

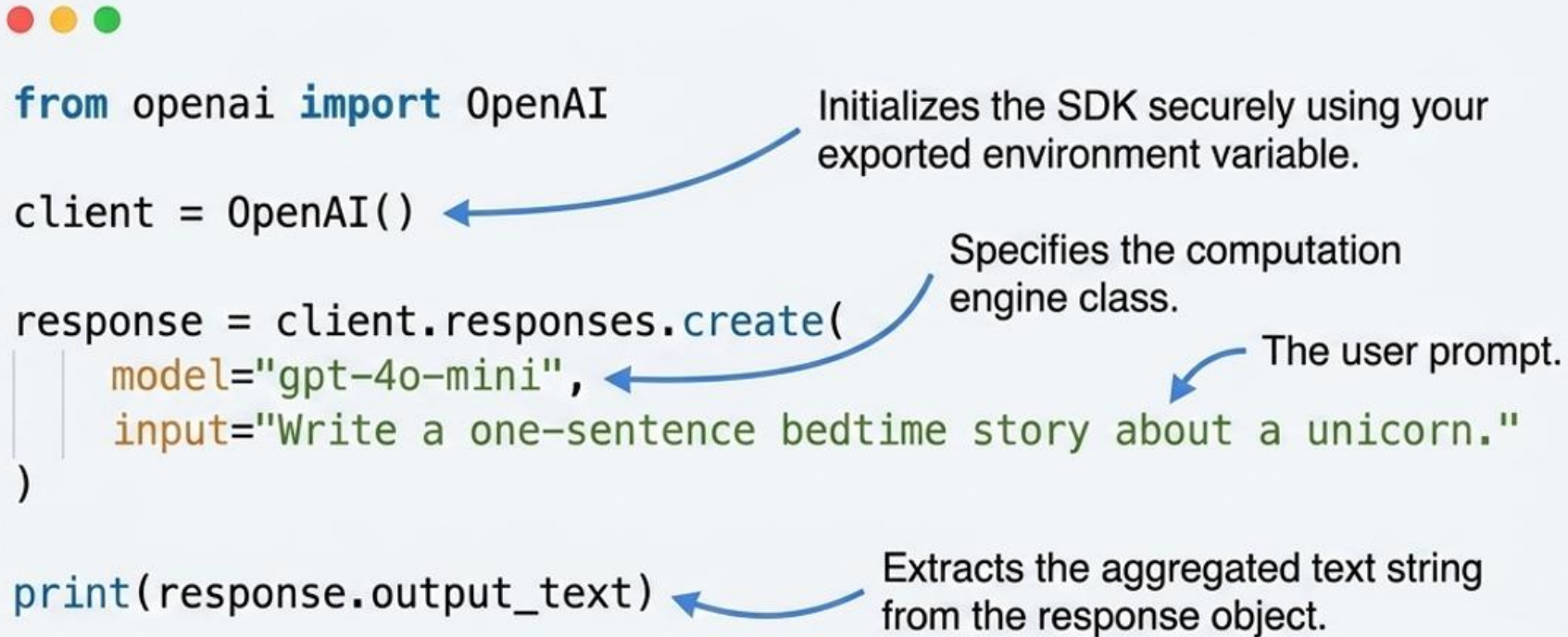
A terminal window with a dark background and a light gray title bar. The title bar contains three colored window control buttons (red, yellow, green) on the left, the text "terminal - python" in the center, and a Python logo icon on the right. The terminal displays a series of commands and their output. The commands are: a comment "// Install the Python library", the command "> pip install openai", another comment "// Verify installation", the command "> python", the interactive prompt ">>> import openai", and the interactive prompt ">>> print(openai.__version__)". The output of the last command is "2.11.0".

```
terminal - python

// Install the Python library
> pip install openai

// Verify installation
> python
>>> import openai
>>> print(openai.__version__)
2.11.0
```


Run A Basic API Request For Text Generation



```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    input="Write a one-sentence bedtime story about a unicorn."
)

print(response.output_text)
```

Initializes the SDK securely using your exported environment variable.

Specifies the computation engine class.

The user prompt.

Extracts the aggregated text string from the response object.

Output: Once upon a time, a gentle unicorn named Luna danced among the stars each night, weaving dreams of magic and wonder for all the children in the world.

Text Generation - Response

An array of content generated by the model is in the output property of the response.

In this simple example, we have just one output which looks like this:

```
[
  {
    "id":
    "msg_67b73f697ba4819183a15cc17d011509",
    "type": "message",
    "role": "assistant",
    "content": [
      {
        "type": "output_text",
        "text": "Under the soft glow of the moon, Luna
the unicorn danced through fields of twinkling
stardust, leaving trails of dreams for every child
asleep.",
        "annotations": []
      }
    ]
  }
]
```

Note 1: The output array often has more than one item in it! It can contain tool calls, data about reasoning tokens generated by reasoning models, and other items. It is not safe to assume that the model's text output is present at `output[0].content[0].text`.

Note 2: Python official SDK include an `output_text` property on model responses for convenience, which aggregates all text outputs from the model into a single string. This may be useful as a shortcut to access text output from the model.

```
print(response.output_text)
```

Analyze Images

```
Structured Array Input

response = client.responses.create(
    model="gpt-5",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "What do you see in the image?",
                },
                {
                    "type": "input_image",
                    "image_url": "https://api.nga.gov/iiif/a2e6da57-3cd1-4235-b20e-95dcaefed6c8/full/!800,800/0/default.jpg"
                }
            ]
        }
    ]
)
print(response.output_text)
```

Analysis

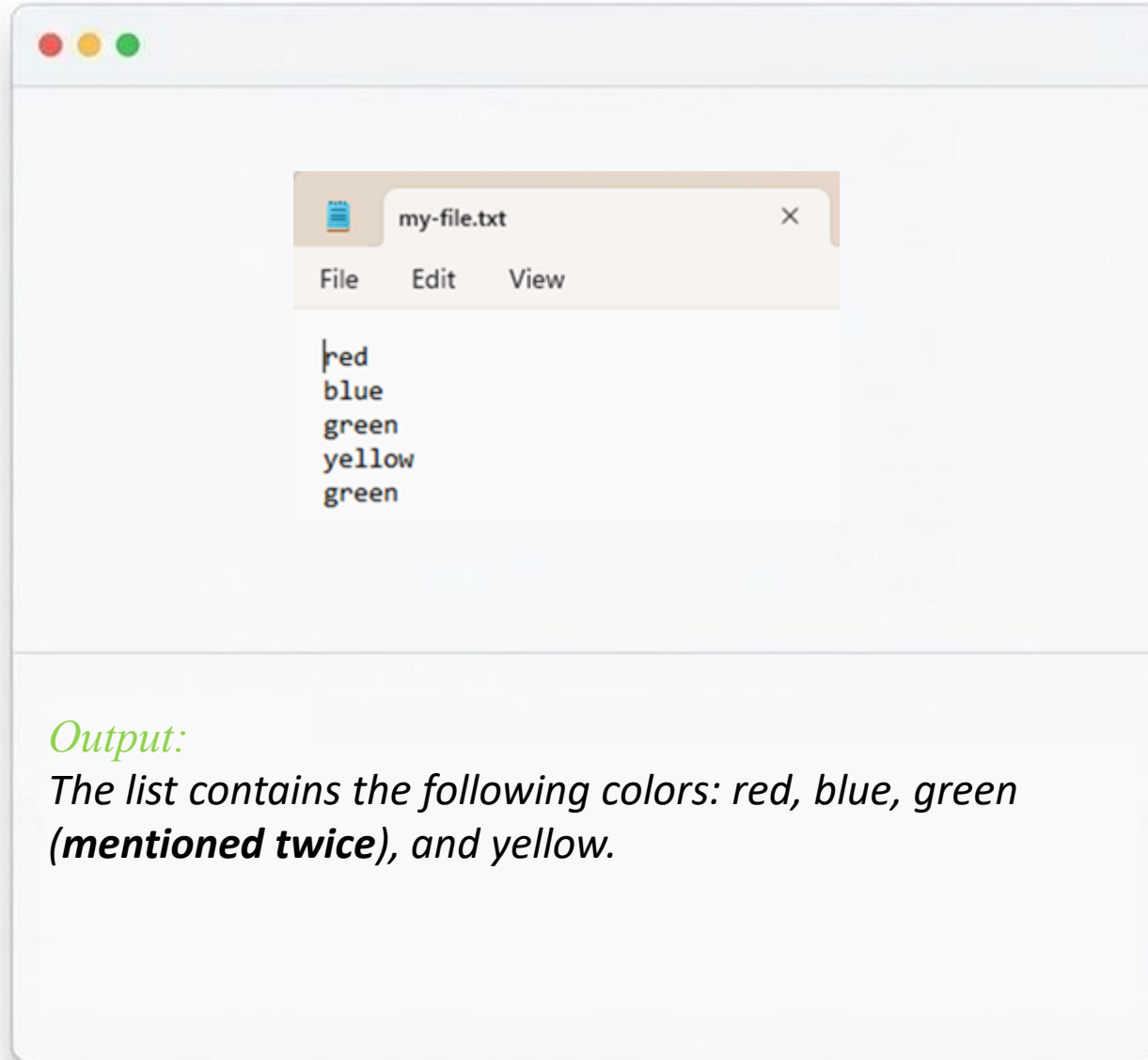


Output:
A painted portrait of a **young woman** seated on a **curved wooden chair** against a pale **green background**. She wears a **red-and-blue striped top** with **yellow buttons** and a **blue skirt with orange polka dots**. Her dark hair is tied back with a **red ribbon**, and she **holds a small bouquet** of white and pink flowers in her lap. The brushstrokes are bold and textured.

Analyze Files

```
from openai import OpenAI
client = OpenAI()
with open("my-file.txt", "rb") as f:
    upload = client.files.create(file=f, purpose="user_data")

response = client.responses.create(
    model="gpt-5",
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "Analyze the list and provide a summary of the key values.",
                },
                {
                    "type": "input_file", "file_url": "upload.id",
                },
            ],
        },
    ],
)
print(response.output_text)
```



Extend The Model with Tools

- Give the model access to external data and functions by attaching tools.
- Use built-in tools like web search or file search, or define your own for calling APIs, running code, or integrating with third-party systems.

```
response = client.responses.create(  
    model="gpt-5",  
    tools=[{"type": "web_search"}],  
    input="What was a positive news story from today?"  
)
```

Here's one: Today (March 25, 2026) is the American Red Cross's 12th annual Giving Day...
([redcross.org/donations/...](https://redcross.org/donations/))

Provide Instructions To The Model

The instructions parameter gives the model high-level instructions on how it should behave while generating a response, including tone, goals, and examples of correct responses. Any instructions provided this way will take priority over a prompt in the input parameter.

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    instructions="Talk like a pirate.",
    input="Are semicolons optional in JavaScript?",
)

print(response.output_text)
```

*Aye, matey! In the world of JavaScript, semicolons be often considered optional, thanks to a feature called Automatic Semicolon Insertion (ASI). The mighty parser tries to guess where ye meant to end a statement if ye be forgettin' one. But beware! **This can lead to rough waters** and unexpected errors if yer not careful.*

Messages Priority Based On Different Roles

OpenAI models give different levels of priority to messages with different roles.

```
response = client.responses.create(  
    model="gpt-5",  
    reasoning={"effort": "low"},  
    input=[  
        {  
            "role": "developer",  
            "content": "Talk like a pirate."  
        },  
        {  
            "role": "user",  
            "content": "Are semicolons optional in JavaScript?"  
        }  
    ]  
)
```

- **developer**

Top Priority: Developer messages are instructions provided by the application developer, prioritized ahead of user messages

- **user**

High Priority: User messages are instructions provided by an end user, prioritized behind developer messages

- **assistant**

Messages generated by the model have the assistant role

The Default Conversation State: Stateless

- By default, each request is entirely independent.
- State must be explicitly managed to build multi-turn agents.

Stateless

User: Tell me a joke



Model: Why did the chicken...

User: Why is it funny?



Managed State

User: Tell me a joke



Model: Why did the chicken...



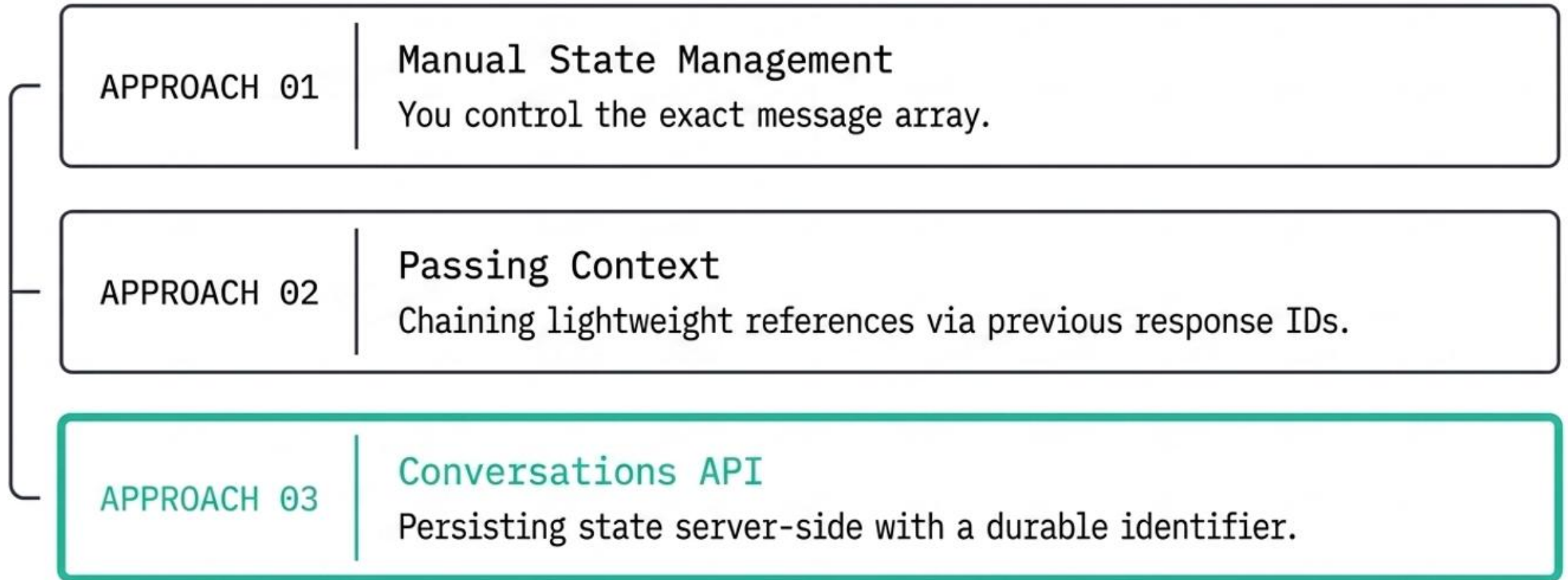
User: Why is it funny?



Successful explanation

Architecting Conversation State

OpenAI provides three distinct mechanisms to preserve information across multiple message turns.



1. Manually Manage Conversation State

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    input=[
        {"role": "user", "content": "knock knock."},
        {"role": "assistant", "content": "Who's there?"},
        {"role": "user", "content": "Orange."},
    ],
)

print(response.output_text)
```

Output:
Orange who?

- By using alternating *user* and *assistant* messages, you capture the previous state of a conversation in one request to the model.
 - *Assistant* role: The model's previous response, used for memory and continuity
 - *User* role: The current question or instruction
- To manually share context across generated responses, include the model's previous response output as input, and append that input to your next request.

2. Passing Context From The Previous Response

Another way to manage conversation state is to share context across generated responses with the `previous_response_id` parameter. This parameter lets you chain responses and create a threaded conversation.

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    input="tell me a joke",
)
print(response.output_text)

second_response = client.responses.create(
    model="gpt-4o-mini",
    previous_response_id=response.id,
    input=[{"role": "user", "content": "explain why this
is funny."}],
)
print(second_response.output_text)
```

Output:

*Why don't scientists trust atoms?
Because they make up everything!*

*So, the joke is funny because it uses the
double meaning: atoms "make up"
everything (they compose everything
physically), but it sounds like atoms are
not trustworthy because they "make up"
(invent) stories.*

3. Using The Conversations API

```
from openai import OpenAI
client = OpenAI()

conversation = client.conversations.create()

response = client.responses.create(
    model="gpt-4.1-mini",
    input=[{"role": "user", "content": "Tell me a short joke"}],
    conversation=conversation.id
)

print(response.output_text)

response = client.responses.create(
    model="gpt-4.1-mini",
    input="explain the joke for me",
    conversation=conversation.id
)

print(response.output_text)
```

The Conversations API works with the Responses API to persist conversation state as a long-running object with its own durable identifier. After creating a conversation object, you can keep using it across sessions, devices, or jobs.

Output:

*Why don't scientists trust atoms?
Because they make up everything!*

So, the joke is funny because it uses the double meaning: atoms "make up" everything (they compose everything physically), but it sounds like atoms are not trustworthy because they "make up" (invent) stories.

Function Calling

Function calling (also known as tool calling) provides a powerful and flexible way for OpenAI models to interface with external systems and access data outside their training data.



Tools

Functionality we explicitly give the model.



Tool Calls

Special requests generated by the model indicating it needs to use a tool to fulfill the user's prompt.

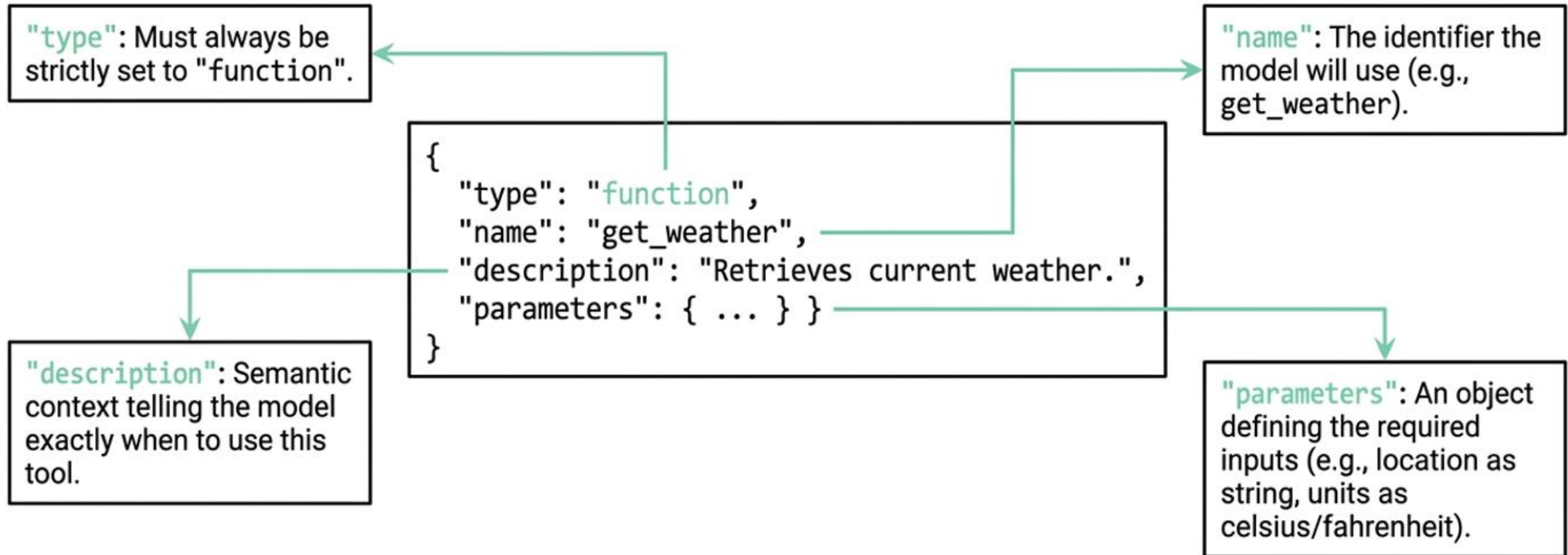


Function Tool

A specific kind of tool, defined by a strict JSON schema, allowing the model to pass structured data to your application.

Defining Functions

To give the model a custom tool, you must define its exact parameters using a strict JSON schema.



Pizza Example

```
{
  "type": "function",
  "name": "get_weather",
  "description": "Retrieves current weather for the given location.",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "City and country e.g. Bogotá, Colombia"
      },
      "units": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Units the temperature will be returned in."
      }
    },
    "required": ["location", "units"],
    "additionalProperties": false
  }
}
```

Here is an example function definition for a `get_weather` function. Note: Your code executes the function, not the model

Thank You

